

Django/MySQL 데이터베이스 ERD 및 코드 전환 가이드

현재 저장 구조를 다시 점검하고, 팀 공용 DB로 옮길 때 필요한 관계형 데이터 모델과 코드 수정 지점을 연결한 실행 문서

한 문장 결론

Agent가 SQLite 파일을 직접 열지 않게 하고, Paper를 중심으로 Django ORM을 통해 MySQL에 저장합니다. PDF는 파일 저장소에 두고, ChromaDB는 벡터 검색 전용으로 유지합니다.

항목	내용
분석 기준	현재 main 브랜치 d07180c
실제 저장 데이터	논문 58편 / 추출 29편, 610섹션 / PDF 21개 / 요약 2건 / 번역 1건
핵심 설계	Paper 중심 정규화 + 사용자 서재 분리 + 작업/대화/출처 추적
권장 구현	Django Service - Repository - ORM - MySQL
작성일	2026-09-16 (Asia/Seoul)

이 문서는 구현 제안서입니다. 아직 Django 모델이나 MySQL 스키마를 실제로 변경하지 않았습니다.

1. 프로젝트를 다시 확인한 결과

현재 프로젝트는 LangGraph Supervisor가 검색, 서재, 다운로드, 추출, 번역, 요약, Deep Search/Research 노드를 순서대로 호출합니다. 기능은 나뉘어 있지만 저장 방식은 SQLite, JSON, Markdown, PDF, ChromaDB, 메모리로 흩어져 있습니다.

기능	현재 저장소	현재 코드	확인된 상태
논문 메타데이터	saved_papers.db + saved_papers.json	src/feature/search.py	papers 58건, SQLite와 JSON 중복
PDF	data/paper_save/*.pdf + downloaded_pdfs.json	src/feature/search_list.py	PDF 21개, 경로 JSON 중복
본문 추출	extracted_papers.db + 일부 Markdown	paper_extractor.py / extractor_tool.py	29편, paper_sections 610건
번역 메인	data/paper_translate/*.md + 체크포인트	translation_tool.py	메인 그래프는 DB 결과 테이블을 쓰지 않음
번역 v2	translate.db + Markdown	translate_tool_v2.py	translations 1건, 별도 흐름
요약 메인	Markdown + ChromaDB + 체크포인트	summary_tool.py	구조화 결과가 MySQL 대상이 아님
요약 v2	summary.db + ChromaDB	summary_tool_v2.py	paper_summaries 2건, 별도 흐름
Deep Search	SQLite 원문 조회 + ChromaDB	deep_search_tool.py	DB 파일 경로에 직접 결합
대화 상태	LangGraph MemorySaver	orchestration/graph.py	프로세스 종료 시 영구 대화 복구 불가

가장 먼저 고쳐야 할 문제

ArxivSearchNode는 검색 결과를 곧바로 save_papers()에 넘깁니다. 그런데 save_papers()의 기본값 extract_content=True 때문에 사용자가 서재에 고르기 전에도 저장과 본문 추출이 실행될 수 있습니다. 검색 결과(Paper 후보)와 사용자 서재 선택(LibraryEntry)을 반드시 분리해야 합니다.

현재 기능 흐름

사용자 요청 -> Supervisor ->

keyword/search/library/download/extract/translate/summarize/deep_search/deep_research -> 응답

현재 영구 저장

SQLite 4개 + JSON 2개 이상 + PDF/Markdown 파일 + ChromaDB. 같은 논문 ID와 경로 정보가 여러 저장소에 반복되어 팀원 PC마다 결과가 달라질 수 있습니다.

2. 설계 원칙: 무엇을 MySQL에 넣고 무엇을 남길까

원칙	결정	이유
한 개의 진실 원천	논문 메타데이터와 처리 결과는 MySQL	JSON/SQLite 복제본의 불일치 제거
Paper 중심	모든 결과가 paper_id FK로 연결	검색, 추출, 번역, 요약, RAG 출처를 한 논문 기준으로 추적
사용자와 공용 데이터 분리	Paper는 공용, LibraryEntry는 사용자별	같은 논문을 여러 사용자가 저장해도 메타데이터 중복 없음
큰 파일 분리	PDF는 Django media 또는 외부 스토리지	MySQL BLOB 부하를 피하고 파일 경로/해시만 관리
벡터 저장소 분리	임베딩은 ChromaDB, 상태는 MySQL	관계형 데이터와 유사도 검색의 역할을 분리
작업 추적	ProcessingJob과 ProcessingChunk	LLM 장기 작업의 진행률, 실패, 재시작을 기록
대화 근거 보존	ChatMessage와 MessageSource	RAG 답변이 어느 논문/섹션에서 왔는지 재현
Agent 저장 금지	Agent -> Service -> Repository -> ORM	에이전트가 저장 기술에 결합되지 않아 테스트/교체가 쉬움

쉽게 비유하면

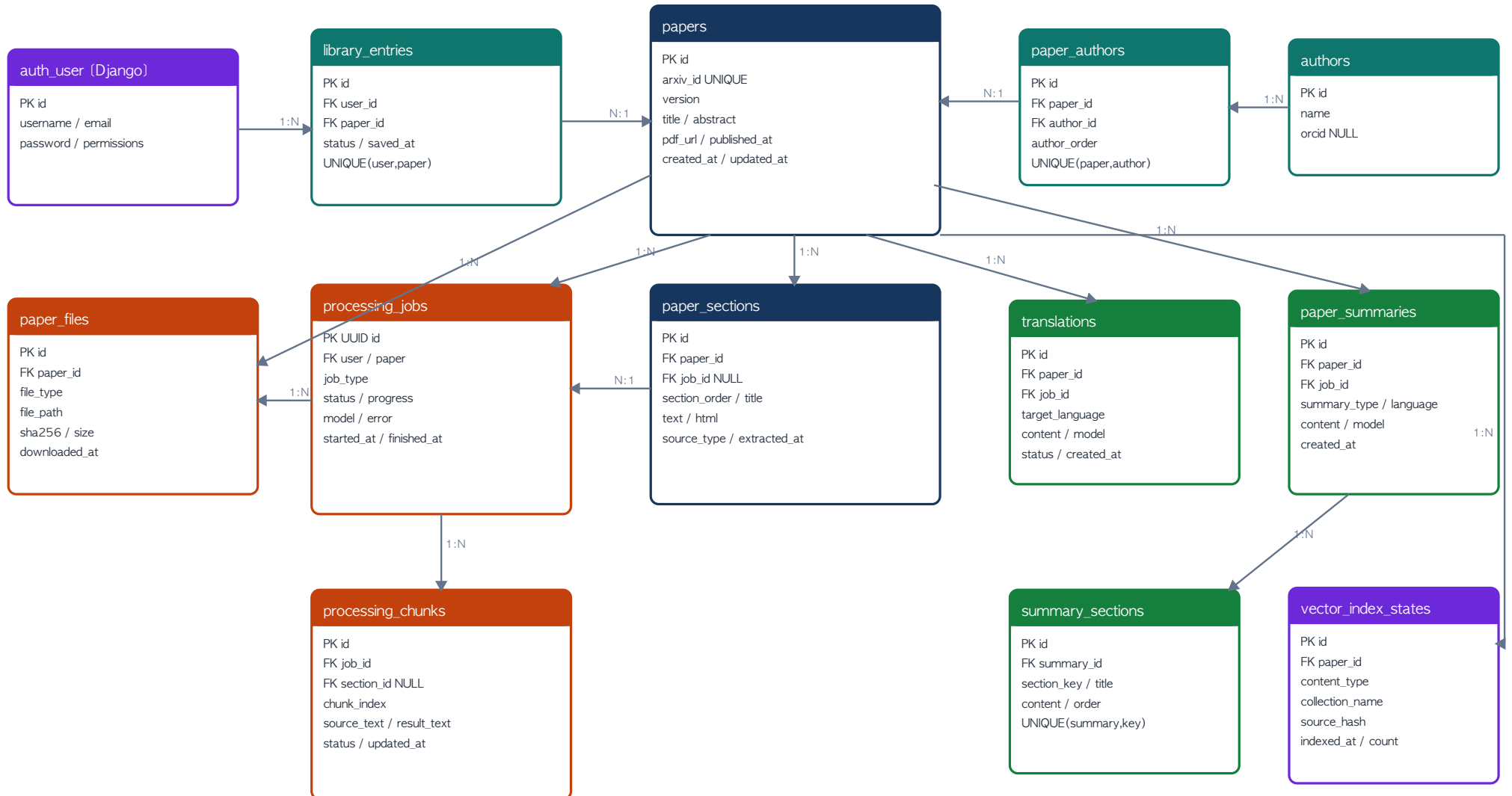
Agent는 주문을 받는 직원, Service는 업무 규칙을 아는 매니저, Repository는 참고 담당자, Django ORM은 통역사, MySQL은 공용 참고입니다. 직원이 참고 파일을 직접 열지 않고 담당자를 통해 요청하는 구조입니다.

저장 대상 구분

- MySQL: 논문, 저자, 서재, 섹션, 번역, 요약, 작업 상태, 대화, 답변 출처, 벡터 인덱스 상태
- Django media 또는 외부 스토리지: 원본 PDF와 필요 시 내보낸 Markdown
- ChromaDB: 본문/요약 임베딩 벡터와 검색용 메타데이터
- LangGraph state: 한 실행 중 필요한 임시 값과 다음 노드 결정

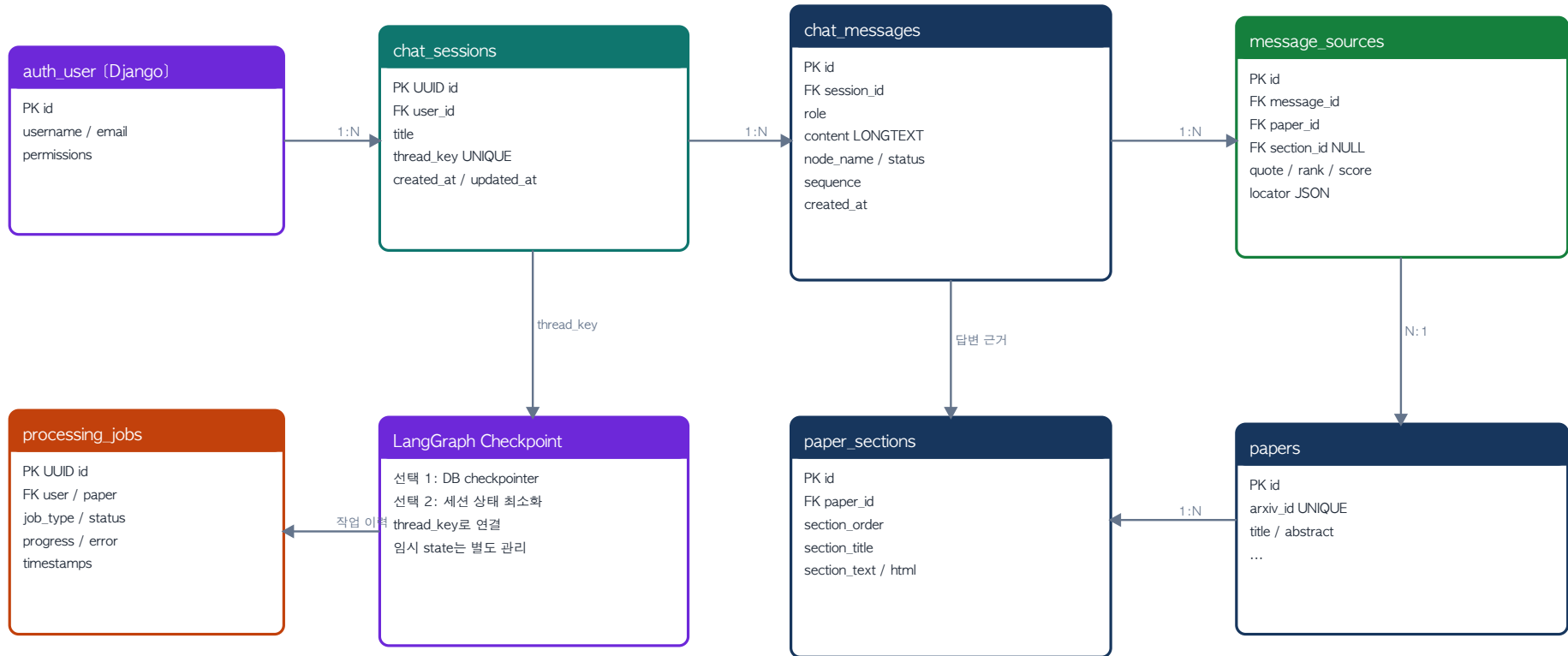
핵심 데이터 ERD - 논문 저장부터 번역, 요약, 검색 인덱스까지

PK는 내부 식별자, FK는 관계 키입니다. arxiv_id는 외부 서비스 식별자로 UNIQUE 처리합니다.



대화 및 RAG 출처 ERD - 어떤 답변이 어느 논문 근거를 사용했는지 추적

LangGraph의 실행 상태 전체를 DB에 복사하지 않고, 사용자 대화와 영구 보존할 결과/출처만 저장합니다.



ChromaDB는 임베딩 벡터를 저장하고, MySQL의 vector_index_states는 인덱싱 여부와 원본 해시만 기록합니다.

3. 최종 권장 테이블과 필요한 이유

테이블	핵심 역할	왜 필요한가	1차/2차
auth_user	Django 사용자/권한	팀원, 일반 사용자, 관리자 권한을 표준 방식으로 관리	Django 기본
papers	논문 공용 원본	모든 기능이 같은 논문 ID와 메타데이터를 참조하는 중심 테이블	1차 필수
authors	저자 사전	문자열 한 칸에 묶인 저자를 검색/필터 가능한 데이터로 분리	1차 권장
paper_authors	논문-저자 N:M	저자 순서와 다대다 관계 보존	1차 권장
library_entries	사용자별 서재	검색 결과와 저장한 논문을 구분하고 사용자별 즐겨찾기/상태 관리	1차 필수
paper_files	PDF 파일 메타데이터	경로 JSON 제거, 다운로드 여부와 파일 해시/버전 추적	1차 필수
processing_jobs	장기 작업 실행 기록	추출/번역/요약/인덱싱 진행률과 실패 원인을 UI에 표시	1차 필수
paper_sections	추출된 본문 단위	Deep Search와 RAG가 논문의 구체적인 근거를 찾는 기준	1차 필수
translations	번역 결과	Markdown 파일만으로는 사용자/논문/모델/상태를 안정적으로 연결할 수 없음	1차 필수
paper_summaries	논문 요약 결과	메인 요약과 v2 요약을 하나의 영구 모델로 통합	1차 필수
summary_sections	요약 구성 항목	핵심 내용, 방법론, 결과, 한계 등 구조화 요약을 개별 조회	1차 권장
processing_chunks	LLM 청크/체크포인트	중단된 번역/요약을 처음부터 다시 하지 않고 재개	2차 권장
chat_sessions	대화방	MemorySaver만으로 사라지는 대화를 사용자별로 복구	2차 필수
chat_messages	사용자/AI 메시지	대화 순서, 노드, 오류 상태를 영구 보존	2차 필수
message_sources	RAG 답변 출처	답변과 Paper/PaperSection을 연결해 근거를 화면에 표시	2차 필수
vector_index_states	벡터 동기화 상태	원문 변경 후 재인덱싱 필요 여부와 Chroma collection 상태 추적	2차 권장

처음부터 전부 만들 필요는 없습니다

1차 목표는 Paper, LibraryEntry, PaperFile, ProcessingJob, PaperSection, Translation, PaperSummary입니다. 로그인도 Django 기본 auth를 사용합니다. 대화/RAG 출처와 청크 체크포인트는 핵심 저장에 안정화된 뒤 붙입니다.

4. 핵심 테이블 필드 설계

테이블	권장 핵심 필드	제약/인덱스
papers	id, arxiv_id, version, title, abstract, pdf_url, published_at, source, created_at, updated_at	UNIQUE(arxiv_id); INDEX(title 일부는 MySQL FULLTEXT 검토)
library_entries	id, user_id, paper_id, status, note, is_favorite, saved_at	UNIQUE(user_id,paper_id); FK user/paper
paper_files	id, paper_id, file_type, file_path, sha256, size_bytes, downloaded_at	UNIQUE(paper_id,file_type,sha256); FK paper
processing_jobs	UUID id, user_id, paper_id, job_type, status, progress, model_name, error_message, timestamps	INDEX(status,job_type); FK user/paper
paper_sections	id, paper_id, job_id, section_order, section_no, title, text, html, source_type, page_start/end, extracted_at	UNIQUE(paper_id,source_type,section_order); FK paper/job
translations	id, paper_id, job_id, source_kind, source_hash, target_language, content, model_name, status, created_at	INDEX(paper,target_language); 동일 입력/언어 중복 정책 결정
paper_summaries	id, paper_id, job_id, source_kind, source_hash, summary_type, language, content, model_name, created_at	INDEX(paper,summary_type,language); 최신 버전 선택 규칙
summary_sections	id, summary_id, section_key, title, content, section_order	UNIQUE(summary_id,section_key); FK summary
processing_chunks	id, job_id, section_id, chunk_index, source_text, result_text, protected_items JSON, status, updated_at	UNIQUE(job_id,chunk_index); FK job/section
chat_sessions	UUID id, user_id, title, thread_key, created_at, updated_at	UNIQUE(thread_key); INDEX(user,updated_at)
chat_messages	id, session_id, role, content, node_name, status, sequence, created_at	UNIQUE(session_id,sequence); FK session
message_sources	id, message_id, paper_id, section_id, quote, rank, score, locator JSON	INDEX(message,rank); FK message/paper/section
vector_index_states	id, paper_id, content_type, collection_name, source_hash, document_count, indexed_at	UNIQUE(paper,content_type,collection_name)

ID 정책

- Django 내부 PK는 BigAutoField를 사용하고 외부 arXiv ID는 arxiv_id UNIQUE 문자열로 둡니다. 외부 ID를 PK로 쓰면 버전 표기와 다른 공급자 확장이 어려워집니다.

- 세션과 장기 작업은 URL 노출과 분산 실행에 유리하도록 UUID를 권장합니다.

삭제 정책

- Paper는 PROTECT 또는 보관 상태를 권장합니다. 논문 삭제로 번역, 요약, 메시지 출처가 연쇄 삭제되는 사고를 막아야 합니다.

- LibraryEntry는 사용자 서재 항목이므로 CASCADE가 자연스럽습니다. MessageSource의 section은 추출 재생성에 대비해 SET_NULL도 검토합니다.

5. 현재 데이터는 어디로 이전할까

현재 데이터	새 테이블/저장소	이전 규칙	현재 수량 검증
saved_papers.db.papers	Paper + Author + PaperAuthor	arxiv 버전 접미사를 분리할지 팀 정책 확정 후 update_or_create	Paper 58건
saved_papers.json	이전하지 않음	Paper 조회로 대체, 중복 인덱스 폐기	DB와 ID 비교만
paper_save/*.pdf	PaperFile + media/papers/	파일 복사 후 sha256, size, 경로 저장	PDF 21개
downloaded_pdfs.json	이전하지 않음	PaperFile에서 역조회	PDF 파일과 대조
extracted_papers.db.paper_sections	PaperSection	paper_id를 Paper FK로 변환, bulk_create	29편 / 610행
paper_translate/*.md	Translation	메타데이터 헤더와 본문을 파싱해 content 저장	메인 번역 산출물 확인
translate.db.translations	Translation	동일 paper/language/source 충돌 정책으로 병합	1건
paper_summary/*.md	PaperSummary + SummarySection	구조화 헤더를 파싱, 원본 파일은 선택적으로 보관	메인 요약 산출물 확인
summary.db.*	PaperSummary / SummarySection / ProcessingChunk	v2 테이블별 변환, temp는 완료 여부 확인 후 처리	요약 2건
vector_db	서버 Chroma 재생성	MySQL 원문/요약을 기준으로 다시 임베딩	collection 문서 수 비교

중요: JSON과 Markdown을 그대로 주 데이터로 옮기지 않습니다

MySQL 구조가 기준입니다. Markdown은 화면 표시나 다운로드 시 DB 내용으로 다시 만들 수 있습니다. 이렇게 해야 DB와 파일 내용이 서로 달라지는 문제를 막을 수 있습니다.

안전한 이전 순서

- 1) Paper -> Author/PaperAuthor -> LibraryEntry -> PaperFile
- 2) PaperSection -> Translation -> PaperSummary/SummarySection -> ProcessingChunk
- 3) 각 단계에서 이전 전/후 개수, 누락된 arxiv_id, FK 오류를 검사
- 4) MySQL을 읽도록 코드를 바꾼 뒤 ChromaDB를 서버에서 재인덱싱
- 5) 충분한 검증 후 SQLite/JSON 쓰기를 중단하고 기존 파일은 읽기 전용 백업

6. 어떤 코드를 고쳐야 하고 왜 고치는가

우선	현재 파일	현재 문제	변경 방향	유지할 기능
P0	requirements.txt	Django/MySQL 드라이버 없음	Django, mysqlclient 또는 PyMySQL, 환경설정 패키지 추가	기존 LangChain 의존성
P0	config/, manage.py	Django 프로젝트가 아직 없음	settings, urls, ASGI/WSGI, DATABASES, media 설정 생성	현재 src 모듈 import 가능하게 구성
P0	src/feature/search.py	SQLite 생성/INSERT, JSON 쓰기, 추출까지 한 클래스에 혼합	검색은 외부 API 결과만 반환. 저장은 PaperService/LibraryService로 이동	arXiv 검색, 의도 분석
P0	src/feature/search_list.py	JSON 검색, SQLite 직접 조회, PDF 경로 JSON 기록	PaperRepository/LibraryRepository 조회, PaperFileService 다운로드	필터, 다운로드 UX
P0	src/feature/paper_extractor.py	메타 DB/추출 DB를 직접 열고 DELETE+INSERT	PaperSectionService.replace_sections() 호출	PDF 파싱/구조 복원 알고리즘
P0	src/tools/extractor_tool.py	두 번째 추출기가 동일 SQLite를 직접 수정	추출기는 섹션 목록만 반환, 저장은 공통 Service가 담당	arXiv HTML 파싱
P0	src/orchestration/adapters.py	검색 노드가 모든 결과를 즉시 save_papers()	검색 후보와 LibraryEntry 생성을 분리하고 반환 ID만 State에 저장	노드 입출력 계약
P1	src/tools/translation_tool.py	Markdown이 최종 원본이며 체크포인트는 파일	TranslationService가 DB 결과와 Job/Chunk를 저장, Markdown은 export	번역 청킹/보호/복원
P1	src/tools/translate_tool_v2.py	summary.db/translate.db에 직접 결합	메인 구현과 통합하거나 legacy import 전용으로 격리	DB 번역 로직 중 필요한 부분
P1	src/tools/summary_tool.py	Markdown + Chroma만 저장	PaperSummaryRepository 저장 후 on_commit으로 벡터 인덱싱	4단 요약 생성
P1	src/tools/summary_tool_v2.py	SQLite 스키마/SQL이 코드 내부에 중복	Django 모델 기준으로 통합, v2는 마이그레이션 어댑터	체크/보호 로직
P1	src/tools/deep_search_tool.py	SQLite read-only 쿼리와 경로에 직접 결합	PaperSectionRepository + VectorSearchService 사용	검색 계약, 근거 랭킹
P1	src/services/fulltext_vector_store.py	SQLite에서 원문을 읽어 Chroma 동기화	Repository에서 원문을 받고 VectorIndexState 갱신	체크 분할/HTML 복원
P1	src/services/summary_vector_store.py	벡터 성공 여부만 Chroma에 존재	인덱싱 성공 후 VectorIndexState 기록	Chroma 저장 구현
P2	src/orchestration/graph.py	MemorySaver만 사용	Django DB 기반 체크포인트 또는 ChatSession/Message 영구 저장 연동	Supervisor 라우팅
P2	src/orchestration/state.py	긴 본문/경로 중심 상태	paper_id, job_id, translation_id, summary_id 중심으로 가볍게 유지	TypedDict 계약
P2	apps/*.py, web_app.py	Streamlit 화면/로컬 파일 직접 읽기	Django View/Template/API로 교체	화면 흐름과 문구 참고

7. 코드 구조는 이렇게 바꾸면 이해하기 쉽다

권장 호출 흐름

Django View/API -> Supervisor/Agent -> Application Service -> Repository -> Django ORM -> MySQL

역할 분담

계층	하는 일	하면 안 되는 일
View/API	로그인, 요청 검증, HTTP 응답	LLM 처리나 SQL 직접 실행
Supervisor/Agent	어떤 기능을 어떤 순서로 부를지 결정	sqlite3.connect, 파일 경로 하드코딩
Service	저장 순서, 중복 정책, 트랜잭션, 권한 같은 업무 규칙	화면 HTML 렌더링
Repository	Paper/Section/Summary 등 데이터 조회/저장 인터페이스	LLM 프롬프트 작성
Django ORM	쿼리 생성, FK, 트랜잭션, Migration	업무 흐름 결정
MySQL	팀 공용 정형 데이터 보존	PDF BLOB, 임베딩 유사도 검색

변경 전

기능 코드가 SQLite를 직접 앎

search.py가 sqlite3.connect(saved_papers.db) -> INSERT -> saved_papers.json 갱신 -> extractor_tool 호출까지 수행합니다.

변경 후

기능 코드는 저장 요청만 함

SearchNode는 검색 결과를 반환하고, 사용자가 저장을 선택하면 LibraryService.save_papers(user, results)가 PaperRepository와 LibraryEntryRepository를 호출합니다. 추출은 별도 ProcessingJob으로 실행합니다.

간단한 코드 모양

```
class PaperRepository:
    def upsert_from_arxiv(self, item): ...
    def get_sections(self, paper_id): ...

class LibraryService:
    @transaction.atomic
    def save(self, user, items):
        paper = repo.upsert_from_arxiv(item)
        LibraryEntry.objects.get_or_create(user=user, paper=paper)

# Agent/Node
ids = library_service.save(user, selected_results)
return {'paper_ids': ids}
```

핵심은 Repository가 꼭 복잡한 클래스로 시작할 필요는 없다는 점입니다. 초반에는 Django QuerySet을 감싼 작은 함수 모음으로 시작해도 되지만, Agent/Tool 파일에서 ORM 호출이 퍼지지 않게 한 곳에 모읍니다.

8. 권장 Django 앱 구조

```
Paper_Scholar/
manage.py
config/
settings.py, urls.py, asgi.py, wsgi.py
apps/
accounts/ # Django auth 확장(필요할 때)
papers/ # Paper, Author, PaperAuthor, PaperFile
library/ # LibraryEntry
processing/ # ProcessingJob, ProcessingChunk
artifacts/ # PaperSection, Translation, PaperSummary, SummarySection
conversations/ # ChatSession, ChatMessage, MessageSource
src/
orchestration/ # Supervisor/StateGraph 유지
services/ # LLM/벡터/도메인 서비스
repositories/ # Django ORM 접근 집중
tools/ # 파싱/번역/요약 알고리즘
templates/ # Django HTML
static/ # CSS/JS/image
media/papers/ # PDF(운영에서는 외부 스토리지 권장)
```

모델 배치 판단

- papers 앱: 외부 논문 자체와 저자, 원본 파일. 다른 모든 앱이 참조하는 중심 도메인
- library 앱: 사용자별 저장 관계. Paper 공용 데이터와 사용자의 선택을 분리
- processing 앱: 비동기 작업과 체크포인트. 추후 Celery/RQ 도입 시 그대로 사용
- artifacts 앱: 추출/번역/요약 결과. 기능 결과가 같은 수명 주기를 가짐
- conversations 앱: 대화와 RAG 출처. 인증 사용자와 연결

MySQL 연결 계정

강사님이 제공한 root는 초기 schema/user/ 권한 생성에만 사용하고, Django는 별도 앱 계정으로 접속합니다.

예: migration용 계정과 CRUD용 계정을 분리하거나, 최소한 root와 애플리케이션 계정을 분리합니다.

비밀번호는 .env에 두고 Git에 올리지 않습니다.

9. 구현 순서와 완료 기준

단계	작업	완료 기준
1. 기반	Django 프로젝트, MySQL app 계정, .env, settings 분리	manage.py check와 DB 연결 성공
2. 핵심 모델	Paper, LibraryEntry, PaperFile, ProcessingJob, PaperSection	Migration 적용 + Admin에서 CRUD 확인
3. Repository	기존 SQLite 접근을 대체할 조회/저장 함수	Repository 단위 테스트 통과
4. 검색/서재	search.py, search_list.py, adapters.py 전환	검색 후보와 내 서재가 구분됨
5. 추출	두 추출기의 저장을 PaperSectionService로 통합	한 논문의 섹션 교체가 원자적으로 동작
6. 번역/요약	Translation, PaperSummary, Job/Chunk 저장	중단/실패/완료 상태가 Admin에서 보임
7. RAG	Deep Search와 Chroma 인덱싱을 Repository 입력으로 전환	답변에 Paper/Section 출처 연결
8. 대화	ChatSession/Message와 체크포인트 정책	서버 재시작 후 대화 복구
9. UI	Streamlit 기능을 Django View/Template/API로 이동	로그인-검색-서재-처리-대화 흐름 완성
10. 이전	관리 명령으로 기존 SQLite/파일 이관	58/29/610/21/2/1 기준 수량 일치

첫 번째 마일스톤

팀 MySQL에서 Paper와 PaperSection을 모든 팀원이 동일하게 조회하고, Django Admin에서 논문/섹션/작업 상태를 확인할 수 있게 만드는 것입니다. 번역, 요약, 채팅은 그 다음에 연결합니다.

테스트해야 할 핵심 상황

- 같은 arxiv_id를 두 번 저장해도 Paper는 한 행이고 LibraryEntry도 사용자별 한 행인가?
- 추출 재실행 시 기존 섹션이 반쪽만 남지 않고 전체가 교체되는가?
- 번역/요약 실패 시 Job은 FAILED이고 오류 메시지가 남는가?
- MySQL 저장 성공 후에만 Chroma 인덱싱을 시작하는가?
- 사용자 A가 사용자 B의 서재/대화를 볼 수 없는가?
- PDF 경로가 사라지거나 해시가 바뀌었을 때 탐지되는가?

10. 구현 전에 팀이 결정할 항목

결정	권장안	결정하지 않으면 생기는 문제
arXiv 버전	arxiv_id는 기본 ID, version 별도 필드	v1/v2가 중복 논문인지 새 버전인지 일관성 없음
번역 기준	본문 번역과 요약 번역을 source_kind로 구분	현재 main과 v2 흐름의 의미가 섞임
요약 기준	PaperSummary를 공통 결과로 두고 summary_type 구분	4단 요약과 v2 단일 요약 충돌
추출 우선순위	PDF 추출을 기본, HTML 추출은 대체/보완	두 추출기가 같은 paper_sections를 덮어씀
결과 버전	source_hash + model_name으로 재생성 여부 판단	원문/모델이 바뀌어도 예전 결과를 최신으로 오인
비동기 작업	1차 동기 + ProcessingJob, 이후 Celery/RQ	처음부터 인프라를 늘리거나 반대로 진행률 추적을 놓침
체크포인트	대화 저장과 LangGraph 내부 체크포인트를 구분	ChatMessage와 그래프 state를 중복 저장
파일 저장	개발 media, 운영 외부 스토리지	여러 서버에서 로컬 파일을 공유하지 못함

최종 체크리스트

영역	확인 질문
DB	Django app 계정과 root 계정을 분리했는가? charset=utf8mb4인가?
모델	FK on_delete, UNIQUE, 인덱스, 시간대가 합의되었는가?
코드	src 내부에서 sqlite3.connect 검색 결과가 legacy importer 외에는 0건인가?
파일	PDF를 BLOB으로 넣지 않고 PaperFile 경로/해시로 관리하는가?
트랜잭션	Paper/Library 저장과 섹션 교체가 transaction.atomic인가?
작업	ProcessingJob의 PENDING/RUNNING/SUCCEEDED/FAILED 상태가 보이는가?
RAG	MessageSource가 답변-논문-섹션을 연결하는가?
보안	.env, DB 비밀번호, API 키, 사용자 PDF가 Git에 포함되지 않는가?
이전	58편, 29편/610섹션, PDF 21개, 요약 2건, 번역 1건을 비교했는가?

최종 판단

이 설계의 목적은 테이블을 많이 만드는 것이 아니라, 기능 코드가 저장 방식에서 독립되게 하는 것입니다. 첫 구현은 핵심 7개 결과 테이블로 시작하고, 대화/출처/체크포인트는 두 번째 단계에서 추가하는 것이 현재 팀 일정에 가장 안전합니다.